

The background is a light blue and green digital landscape. It features several tall, rectangular server racks with glowing green lights and digital readouts. A white robot with a black visor and a small antenna stands in the center, gesturing towards the right. The floor is composed of a network of glowing green lines and dots, resembling a circuit board or data flow diagram. The overall aesthetic is futuristic and technological.

Data Creation

A (VERY) high-level overview

Seminar Structure

1. What is data creation?
2. What makes good data?
3. LLM-driven data generation; a high-level overview
4. Why not just use the internet?
5. DataComp-LM
6. Curation is subtractive design + FineWeb
7. From curation to synthesis
8. Terminology Clarifications
9. The Pile
10. Synthesis #1: Pretraining (WRAP)
11. Synthesis #2: Post-training (Self-Instruct and Evol-Instruct)
12. Student-Teacher paradigm
13. Model Collapse (Shumailov)
14. Can we avoid model collapse? (Gerstgrasser)
16. Evaluation
17. Benchmark Decontamination
18. LLM decontaminator
19. Other evaluation methods
20. Open Problems
21. Bibliography

What is Data Creation?

- We're all used to the paradigm of finding data: scraping, licensing, begging. Data creation, as the name suggests, changes this view.
- Data creation is the deliberate design and construction of data to be used for downstream tasks (most popularly now— training models)
- **Why?** Long et al. has some thoughts:
 - data is expensive
 - data is often scarce in domains where it matters the most
 - data is legally constrained
- The field of data creation can be kind of organized as a three-stage workflow:
 - **Generation**: producing/synthesizing candidate data
 - **Curation**: deciding what to keep (filtering, quality scoring, reweighting)
 - **Evaluation**: checking whether whatever we did worked (optional if you're confident enough i guess)

What Makes Good (Generated) Data?

- Outside of the obvious aspects, like cleanliness, spanning the necessary space, being of sufficient quantity, being economically feasible to generate.
- The standard for high-quality data may vary by downstream task, but two requirements in particular stand out as a challenge in current literature:
 - **Faithfulness.** Data must be logically and grammatically coherent. Hallucination and the fat-tailed knowledge distribution of LLMs tend to introduce noise.
 - **Diversity.** Variation in the generated data.
 - It's intractable/impossible to try and estimate the underlying data-generating distribution for every real-world task we seek to model and then sample from it to get accurately diverse data.
 - Due to inherent biases in LLMs, uncontrolled generation of content tends to lead to monotonicity of outputs (even with high decoding temperature)

Better Synthetic Data by Retrieving and Transforming
Existing Datasets

Saumya Gandhi*, Ritu Gala*, Vijay Viswanathan,
Tongshuang Wu, Graham Neubig
Carnegie Mellon University

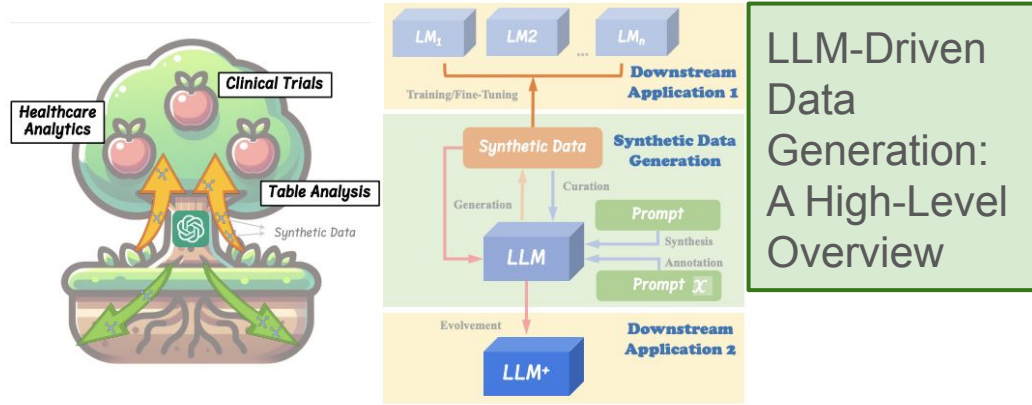


Figure 1: Illustration of the LLMs-based application ecosystem, where synthetic data serves as the flowing nutrients for fruiting (training of small LMs or fine-tuning task-specific LLMs) and rooting (training stronger LLMs or self-improvement).

$$\mathcal{D}_{\text{gen}} \leftarrow \mathcal{M}_p(\mathcal{T}, \mathcal{D}_{\text{sup}}),$$

Formalization of the data generating process. $\mathcal{D}_{\text{gen}}$ is final generated dataset, $\mathcal{M}_p$ is pretrained model $\mathcal{M}$ prompted with p , $\mathcal{T}$ is task, $\mathcal{D}_{\text{sup}}$ is seed data samples

$$p(\mathcal{T}, \mathcal{D}) \leftarrow E(e_{\text{task}}, e_{\text{condition}}, e_{\text{demo}})$$

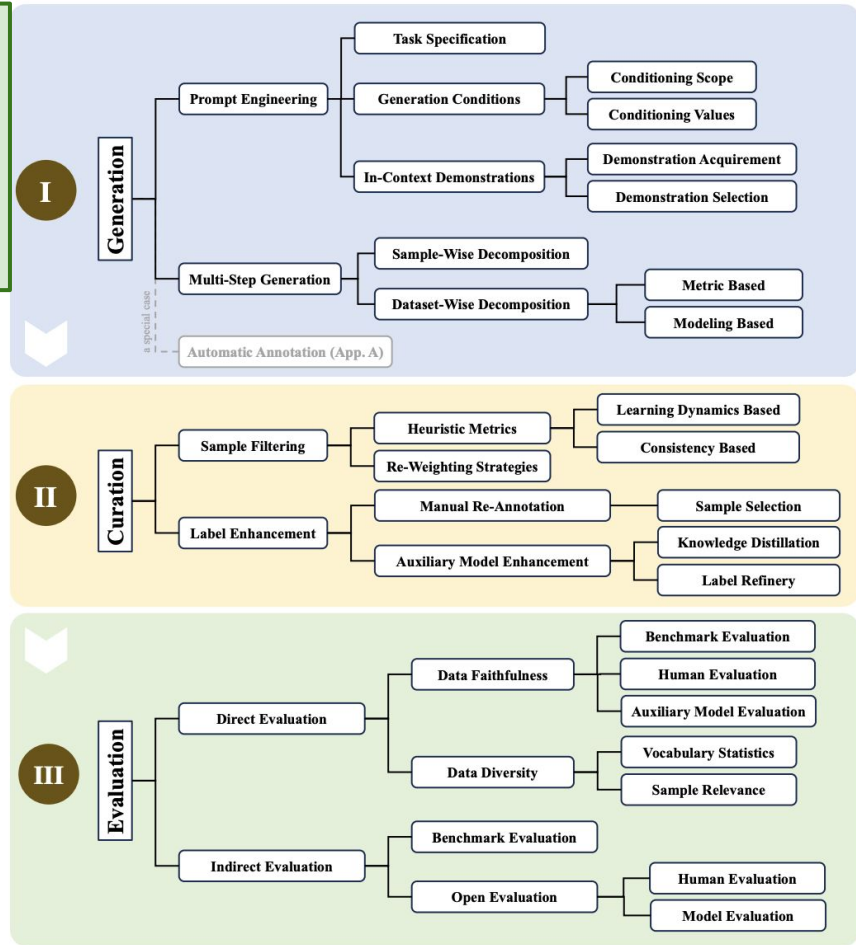


Figure 2: A taxonomy of LLMs-driven synthetic data generation, curation, and evaluation.

A stylized illustration of a robot standing in a digital environment. The robot is white with a black visor and is positioned in the center. It is surrounded by several server racks that are glowing with a greenish-blue light. The background is a light blue gradient with glowing lines and dots, suggesting a network or data flow. The overall aesthetic is futuristic and technological.

Curation

Why generate data when you can use what already exists?

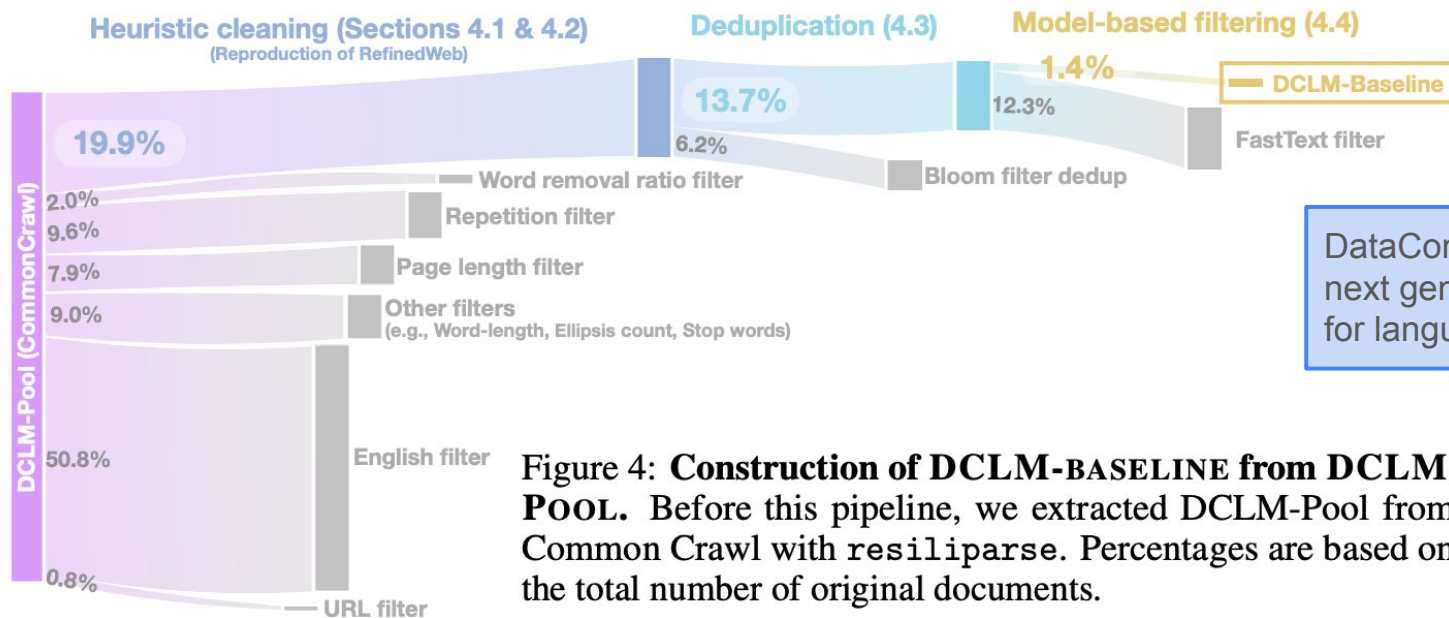
Why Not Just Use the Internet?

- Fair question. The internet has hundreds of zettabytes (a billion terabytes) of data. Why not use that?
- Common Crawl is a free and open repository of web crawl data. 300 billion pages spanning 19 years. Cited in thousands of research papers. 3-5 billion new web pages every month. Essentially every open pretraining corpus is a filtered view of it (FineWeb, DCLM).
- Without it, pretraining research would be restricted to organizations that own a search crawler.
- Research finds that even amongst the hundreds of trillions of tokens' worth of data that can potentially be extracted from the internet, only a small percentage survives a proven curation strategy.



<https://www.datacomp.ai/dclm/>

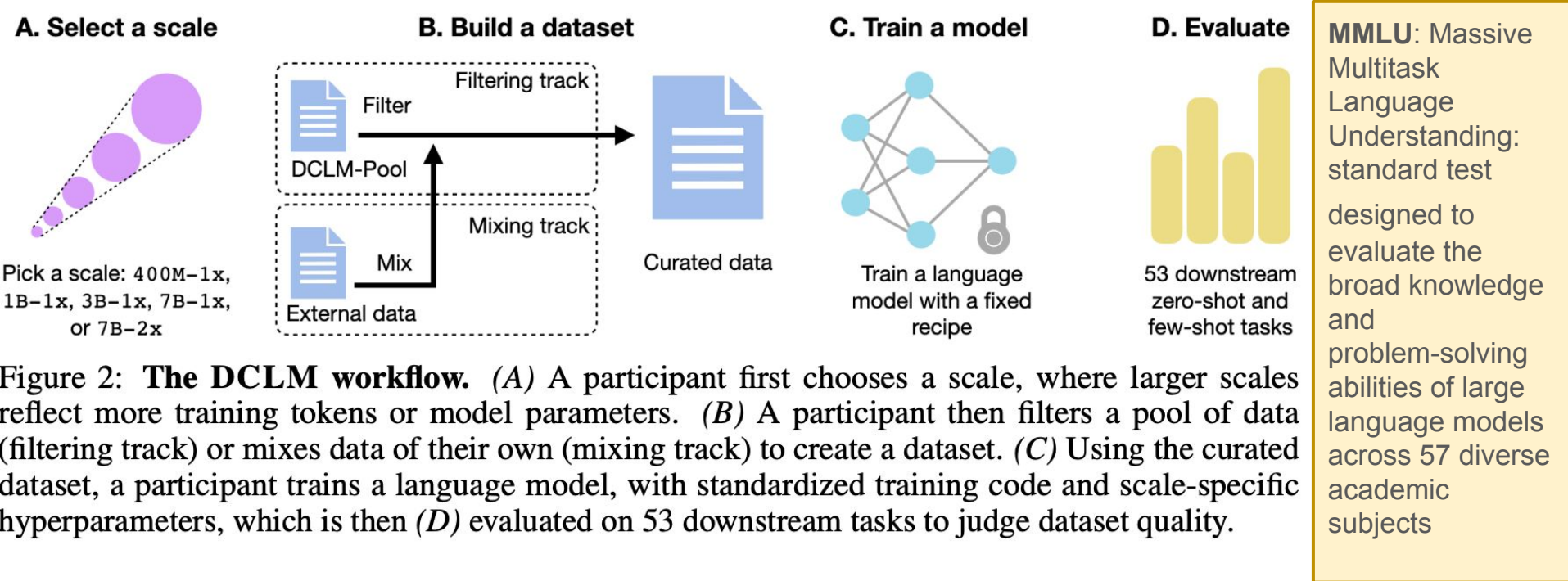
DataComp-LM: The Internet Data Curation Attempt



DataComp-LM: In search of the next generation of training sets for language models

Figure 4: **Construction of DCLM-BASELINE from DCLM-POOL.** Before this pipeline, we extracted DCLM-Pool from Common Crawl with `resiliparse`. Percentages are based on the total number of original documents.

- DCLM is a testbed for controlled dataset experiments aimed at improving LLMs
- Standardized corpus of 240T Common Crawl tokens was provided (DCLM-Pool). After filtering pipeline, the baseline training set is around 3.8T tokens. Roughly **98%** of the pool is discarded before the model ever sees it.



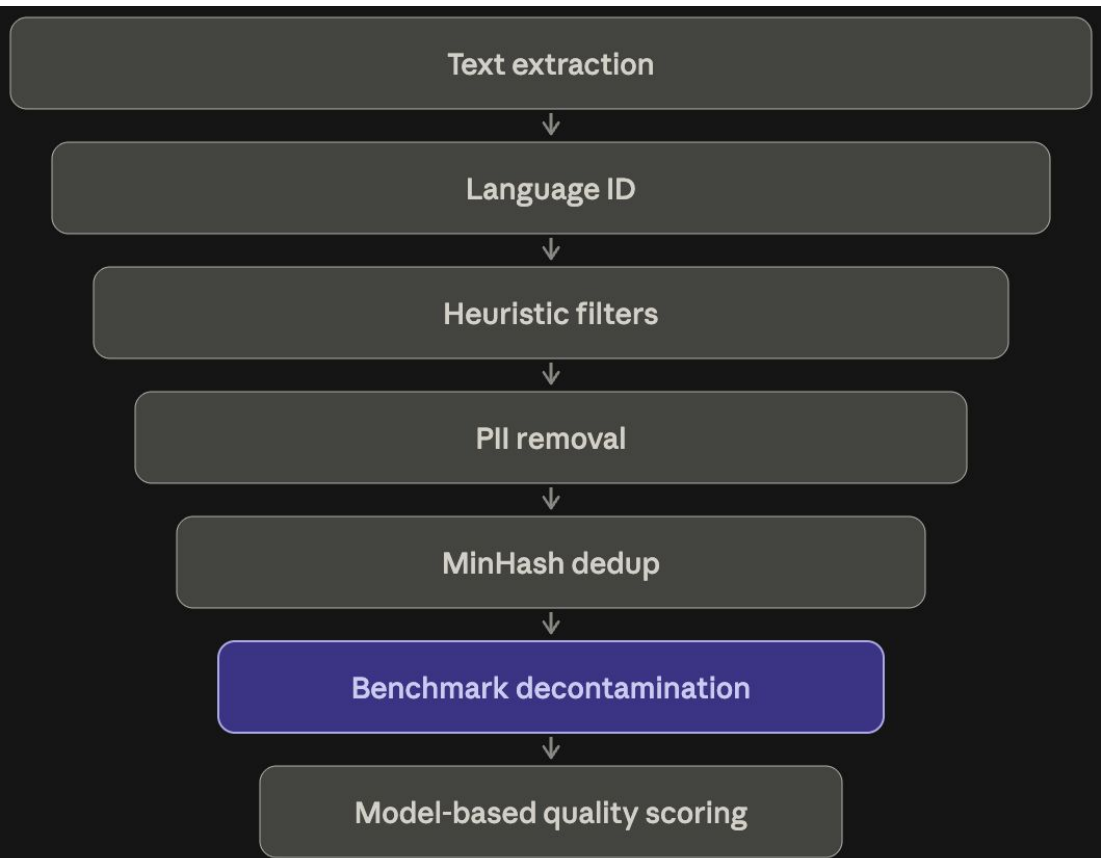
How do we know
the filtering worked?
Train a model and
score it on
benchmarks.

				CORE	MMLU	EXTENDED
FineWeb edu	7B	0.14T	✓	38.7	26.3	22.1
FineWeb edu	7B	0.28T	✓	41.9	37.3	24.5
DCLM-BASELINE	7B	0.14T	✓	44.1	38.3	25.0
DCLM-BASELINE	7B	0.28T	✓	48.9	50.8	31.8
DCLM-BASELINE + StarCoder + ProofPile2	7B	2.6T	✓	57.1	63.7	45.4

Curation is Subtractive Design

The FineWeb Datasets:
Decanting the Web for the
Finest Text Data at Scale

What does that 98% discarding procedure actually look like?



WARC (Web ARChive format) files from Common Crawl parsed with trafilatura. Don't blindly trust a pre-cleaned version!

Classifier scores document language. Confidence threshold for English. URL blocklist thrown in as well.

Rules derived from comparing stats of high and low quality data (ex: fraction of lines ending in some punctuation)

Legal/ethical step. Regex scrubbing of emails and IP addresses (Personally Identifiable Information)

Deduplication step. Not a monotonic relation with quality.

Removing training documents that contain evaluation items. Train and test set separation.

Llama-3-70B-Instruct annotates sampled documents for educational value on a 0–5 scale, discarding low scores

MinHash: A Brief Note

Jaccard Similarity Coefficient

• Commonly used as an indicator for set similarity, also known as "Jaccard Index"

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

$A = \{1, 2, 3, 4, 5, 6\}$

$B = \{1, 3, 5, 7\}$

$|A \cap B| / |A \cup B| = |\{1, 3, 5\}| / |\{1, 2, 3, 4, 5, 6, 7\}| = 3 / 7$

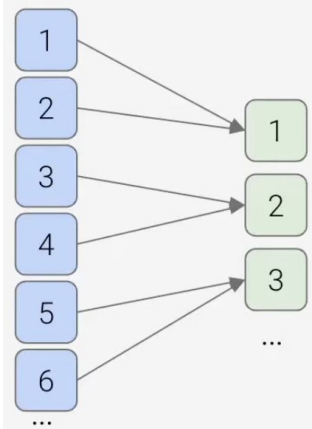
We convert each document to a smaller signature using a hash function H . Let's consider two documents, d_1 and d_2 .

1. We assume that the signature is small enough to fit in memory
2. If $\text{similarity}(d_1, d_2)$ is "high", then $P(H(d_1) == H(d_2))$ is "high"
3. If $\text{similarity}(d_1, d_2)$ is "low", then $P(H(d_1) == H(d_2))$ is "low"

The methodology of achieving this is to approximate the Jaccard similarity coefficient.

To efficiently determine the Jaccard similarity of two documents, we construct a series of permutations.

Locality Sensitive Hash



We have several billion documents, and we want to identify near-duplicate documents.

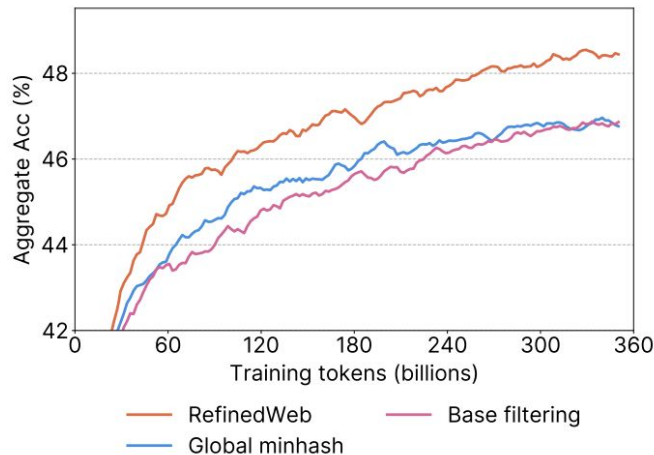
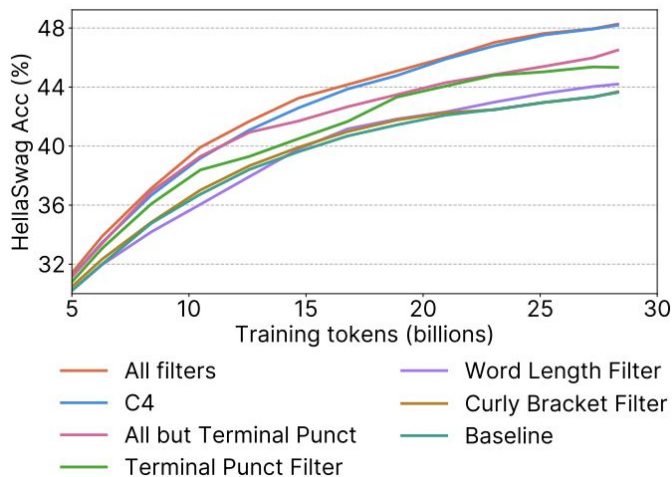
How can we solve this efficiently?

If we do the naïve approach, we would go through each pair-wise document in the collection, but that is too slow.

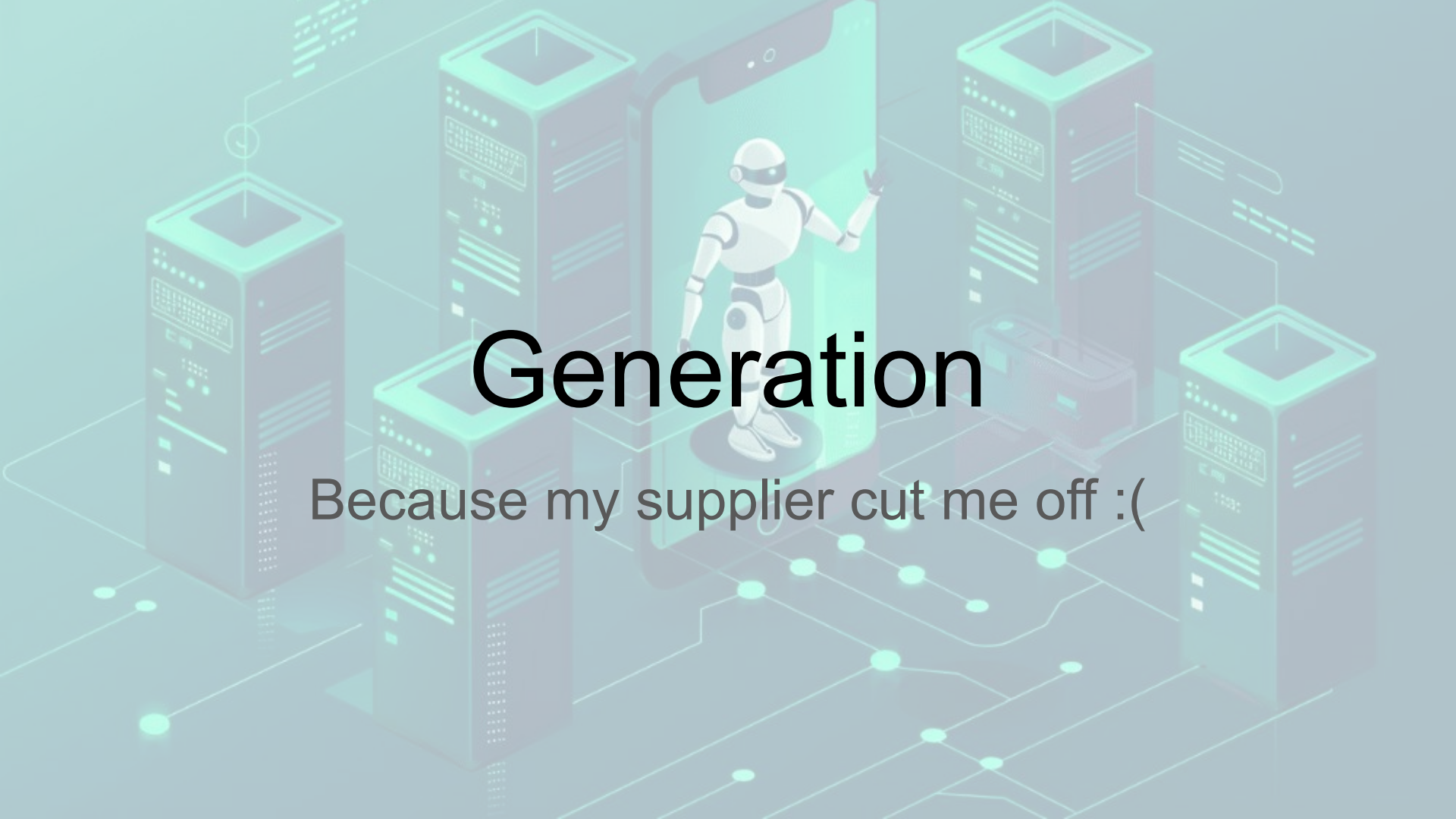
We need an efficient way to cluster similar documents -- locality sensitive hashing!

From Curation to Synthesis

- We've seen the pure curation attempts, and also seen why curation is necessary. The other side we must pivot to now is actual generation and synthesis of data.
- Until now, we spoke purely about deciding what data trains the models, and coming up with methods and techniques to arrive at this final set of data.
- Now, we move to a model itself writing the data that trains models.
- Generation itself can be split as being applied to two stages of a traditional LLM pipeline: **pretraining** and **post-training**.



Visuals from FineWeb illustrating the diminishing returns of more training tokens on the same model (perf measured on HellaSwag Benchmark)



Generation

Because my supplier cut me off :(

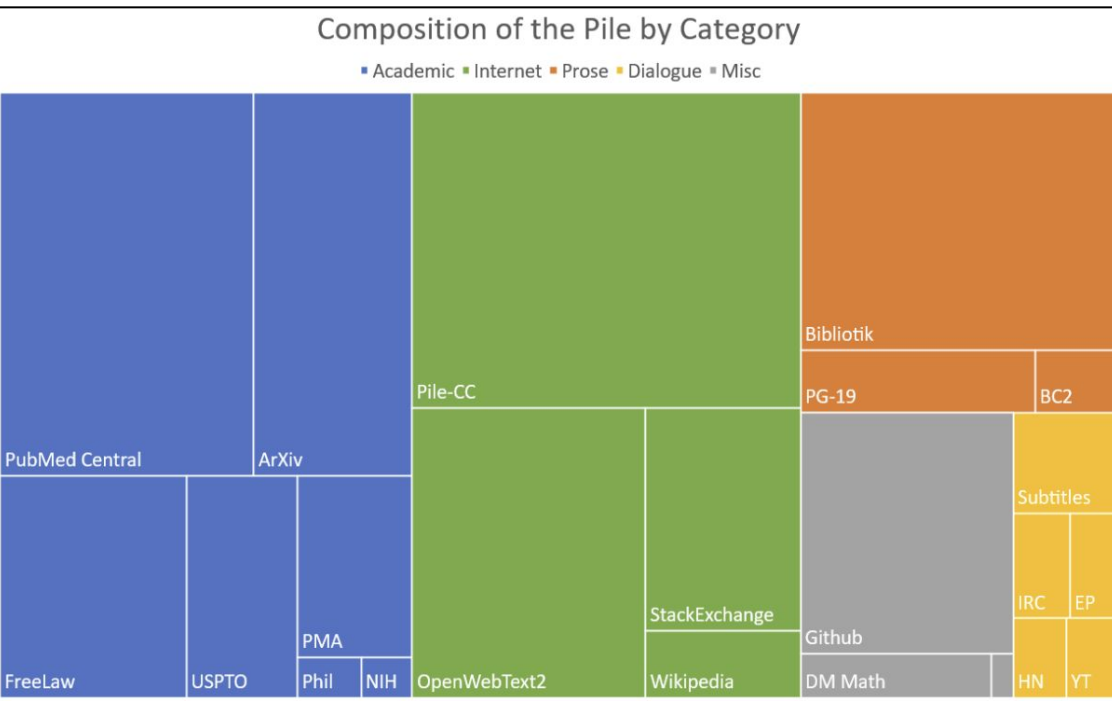
Terminology Clarifications

- **Pre-training:** singular task of next-token prediction. Self-supervised. Product of this stage is a model that can continue text but can't follow instructions or hold a conversation.
- **Post-training:** second phase of development where a raw, pre-trained base model is refined using curated datasets and feedback to safely follow instructions, hold conversations, and perform specific tasks. Supervised.
- **Average zero-shot accuracy:** take a suite of downstream tasks (question answering, commonsense, reading comprehension), give the model each question with no worked examples in the prompt, score it, average across tasks. "Zero-shot" means no demonstrations. Test the pretraining, not the in-context learning ability.
- **Perplexity:** exponential of average negative log-likelihood per token. Informally, how many equally likely options the model was effectively choosing between at each step. Lower is better; a perplexity of 10 means it was about as uncertain as picking uniformly among 10 tokens.
- **C4 Dataset:** heuristic filterings of a 2019 crawl. Dropping lines without a terminal punctuation mark, that mentioned javascript, or that had "terms of use"/"cookie policy" statements, and dropping documents that were too short or that contained "lorem ipsum" or a curly bracket ({}).

The Pile: A Brief Note

The Pile: An 800GB Dataset of Diverse Text for Language Modeling

- 825.18 GiB English text dataset designed for training large scale language models. The Pile is composed of 22 diverse and high-quality datasets, including both established natural language processing datasets and several newly introduced ones. In addition to its utility in training large language models, the Pile can also serve as a broad-coverage benchmark for cross-domain knowledge and generalization ability of language models.

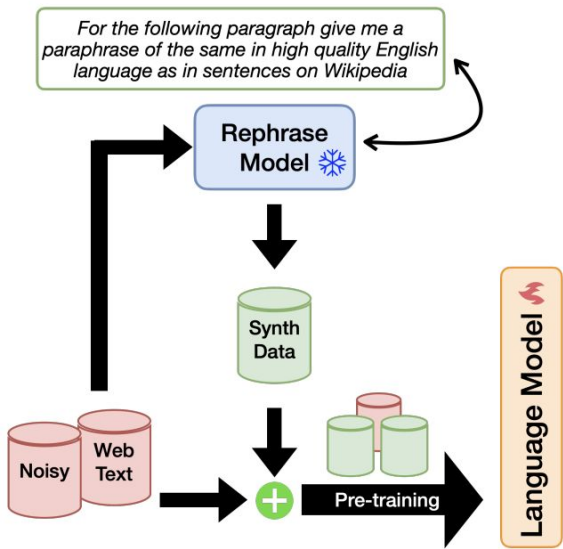


- Specifically curated to teach Large Language Models (LLMs) foundational language, logic, and broad cross-domain knowledge during their initial pretraining phase.
- I introduced it here because it's used as a perplexity evaluation during pretraining (it forms the test set) in the WRAP paper that we're going to cover in the next slide.

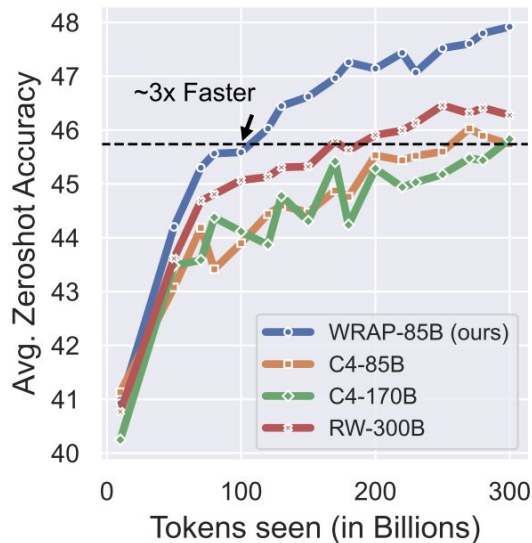
Synthesis #1: Pretraining

Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling

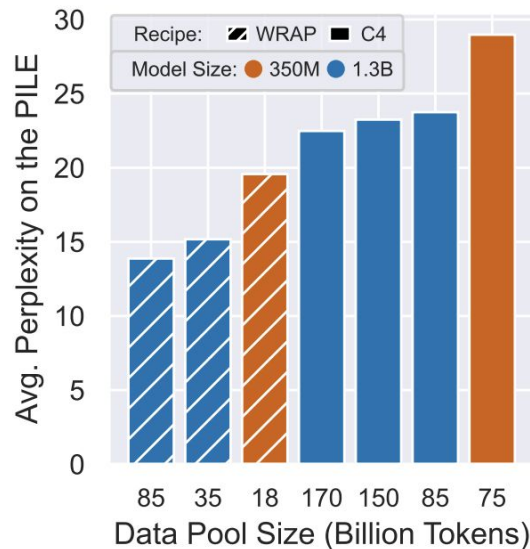
- **Core idea:** rewrite what you'd usually discard. Web scrapes are noisy and unstructured.
- **WRAP** (Web Rephrase Augmented Pretraining): prompt an off-the-shelf instruct model to paraphrase web text into a target style (“like Wikipedia”, “Q&A format”, etc.)
- Pretrain the LLM on a mixture of real and synthetic data.
- Rephrased + real data beats either one of them alone.



(a)

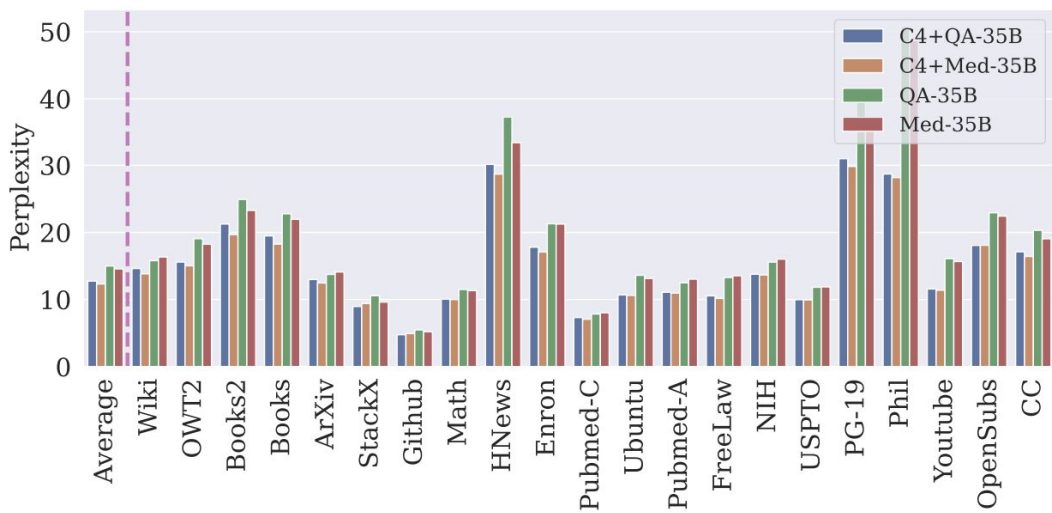
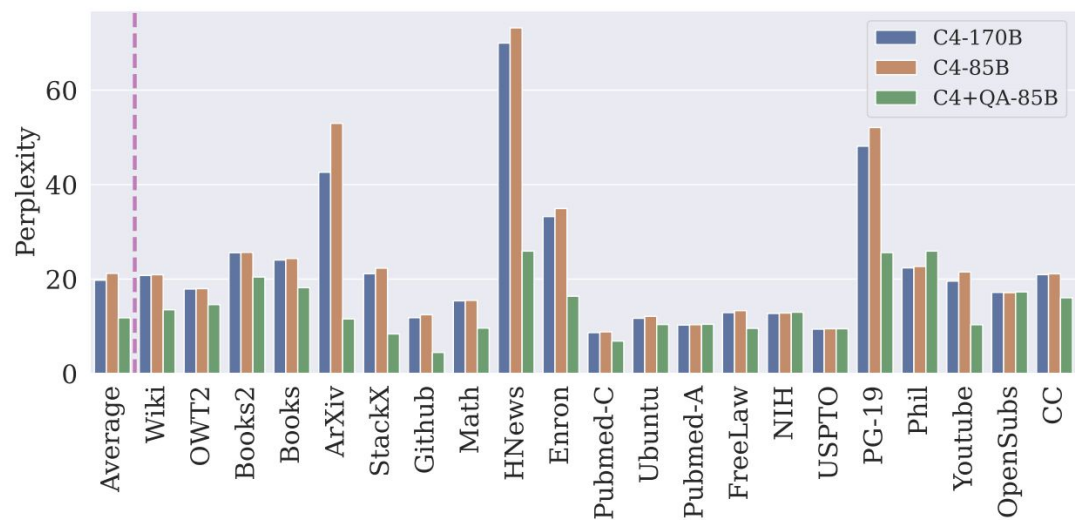


(b)



(c)

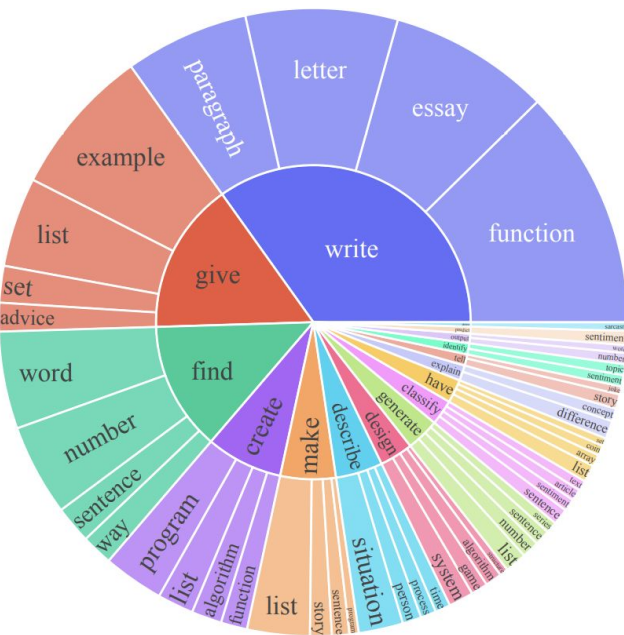
- Jointly train the LLM on real and synthetic data. Investigate the impact of the rephrasing on model performance.
- Re-phrased synthetic data has higher utility (fewer tokens suffice) than just real data because it:
 - incorporates style diversity that closely reflects downstream evaluation style.
 - has higher signal density. Raw web text is padded with boilerplate, typos and malformed structure. A cleaned paraphrase carries the same content in fewer, better-formed tokens



Synthesis #2: Post-Training

SELF-INSTRUCT: Aligning
Language Models with
Self-Generated Instructions

- **Core idea:** you don't have a corpus to rewrite, so grow one from a seed.
- **Self-Instruct:** start with a few hundred human-written tasks, prompt the model to generate new ones, filter, add back to the pool, repeat.



top 20 most common root verbs (inner circle) and their top 4 direct noun objects (outer circle) in the generated instructions.

Instruction: Given an address and city, come up with the zip code.

Input:

Address: 123 Main Street, City: San Francisco

Output: 94105



Instruction: I am looking for a job and I need to fill out an application form. Can you please help me complete it?

Input:

Application Form:

Name: _____ Age: _____ Sex: _____

Phone Number: _____ Email Address: _____

Education: _____ ...

Output:

Name: John Doe Age: 25 Sex: Male

Phone Number: ...



Instruction: How to write a code for converting degrees fahrenheit to celsius.

Input: Null

Output:

```
def convert_fahrenheit_to_celsius(fahr):  
    celsius = (fahr - 32) * 5 / 9  
    return celsius
```



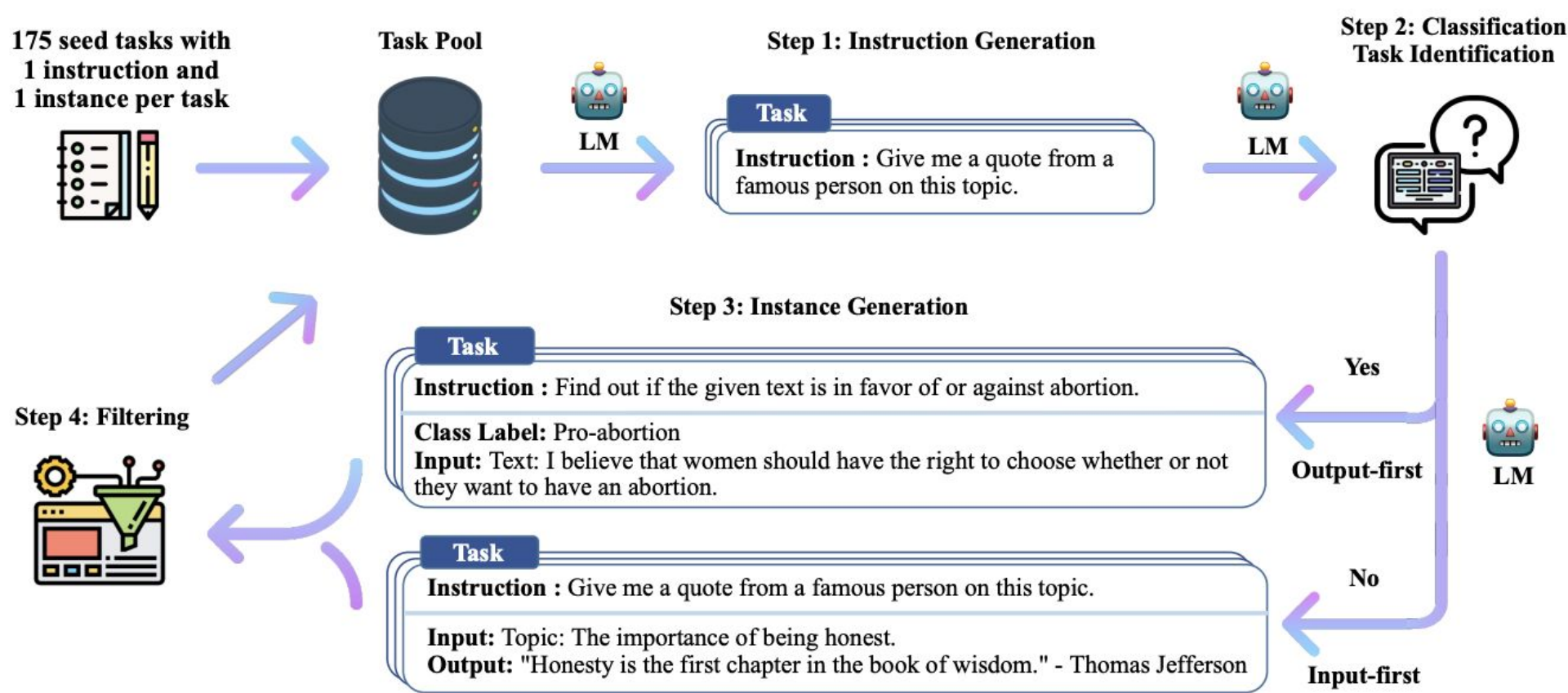
Instruction: Write a letter from the perspective of a cat.

Input: Null

Output:

Dear [Owner], I am writing to you today because I have a problem. You see, I've been living with you for the past [Number of Years] and I love it here. The food is great, the bed is comfy, and there are plenty of toys to play with. But lately, I've noticed that you're not as affectionate towards me as you used to be. In fact, sometimes when I try to cuddle up next to you on the couch, you push me away ...

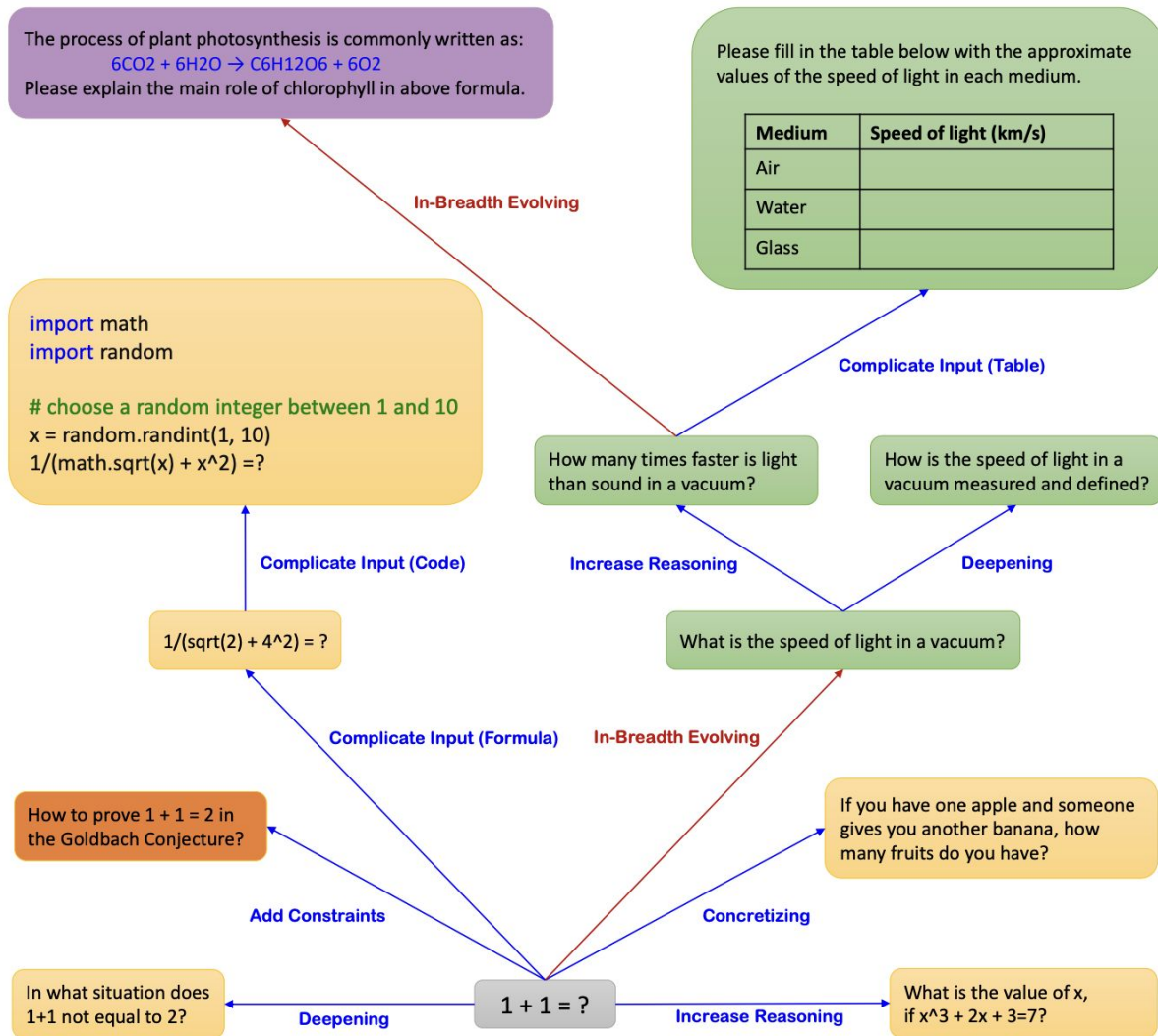




A high-level overview of SELF-INSTRUCT. The process starts with a small seed set of tasks as the task pool. Random tasks are sampled from the task pool, and used to prompt an off-the-shelf LM to generate both new instructions and corresponding instances, followed by filtering low-quality or similar generations, and then added back to the initial repository of tasks. The resulting data can be used for the instruction tuning of the language model itself later to follow instructions better.

- **Evol-Instruct:** Don't sample new tasks. Mutate existing ones along complexity axes: add constraints, deepen, concretize.
- “using LLMs instead of humans to automatically mass-produce open-domain instructions of various difficulty levels, to improve the performance of LLMs.”

WizardLM: EMPOWERING LARGE PRE-TRAINED LANGUAGE MODELS TO FOLLOW COMPLEX INSTRUCTIONS



The Student Inherits the Teacher

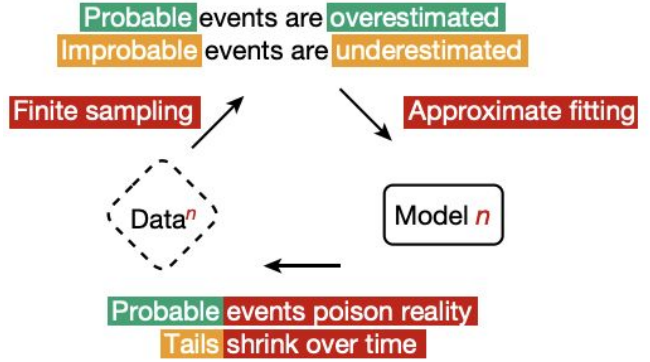
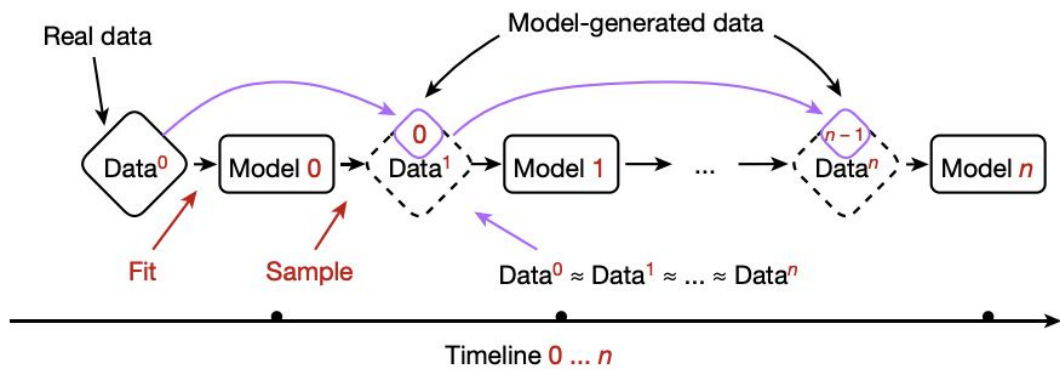
Absolute Zero,
R-zero

- Every pipeline on the last three slides has one shape: **a strong model writes the data for a weaker one.**
- WRAP has Mistral rephrasing text to train a 1.3B model. Self-Instruct has GPT-3 writing tasks. Evol-Instruct has ChatGPT writing responses that fine-tune LLaMA.
- The student inherits the teacher's formatting, refusal style, and length distribution. Known, and mostly treated as cosmetic.
- Two things it inherits that aren't cosmetic: the teacher's blind spots and the teacher's benchmark exposure.
- The teacher is a ceiling. If your data is the teacher's output, imitation caps you at the teacher.
- Can i do this iteratively/recursively? What if i run inheritance across one generation, then make the student the teacher for another model? This leads to what Shumailov calls model collapse:
 - a teacher's samples under-represent its own tails, so rare content thins with every generation
 - remember our diversity requirement? this kind of destroys it.

Model Collapse: The Failure

AI models collapse when trained on recursively generated data

- Training recursively on model-generated content causes irreversible defects in the resulting models, in which tails of the original content distribution disappear.



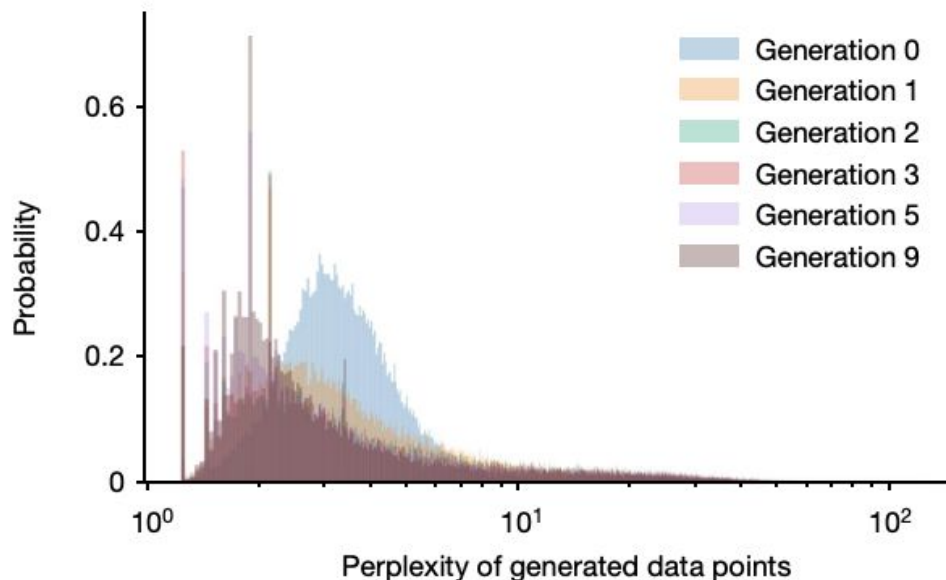
Statistical approximation error:
non-zero probability that information can get lost at every step of resampling. Rare events get dropped by chance. Vanishes as n tends to inf.

Functional expressivity error:
The model class can't represent the true distribution (ex: fitting a Gaussian to bimodal data, or any finite network to an arbitrary density). Negligible with sufficiently expressive models.

Functional approximation error:
The learning procedure itself is imperfect. SGD, regularization, biased objectives. Even with infinite data and unlimited expressivity, the optimizer doesn't land on the truth.

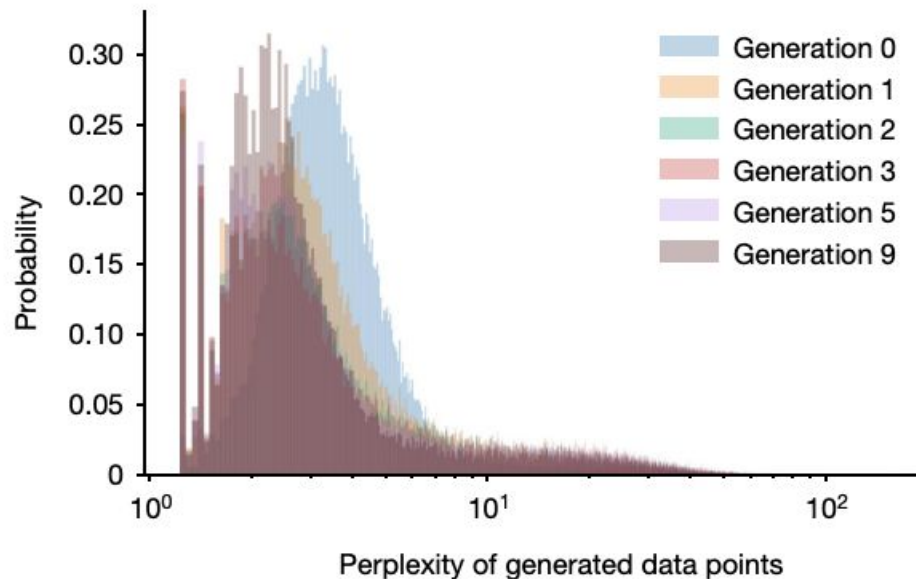
b No data preserved, five epochs

Perplexity of generated data points evaluated by
model trained with real wikitext2



c 10% data preserved, ten epochs

Perplexity of generated data points evaluated by
model trained with real wikitext2



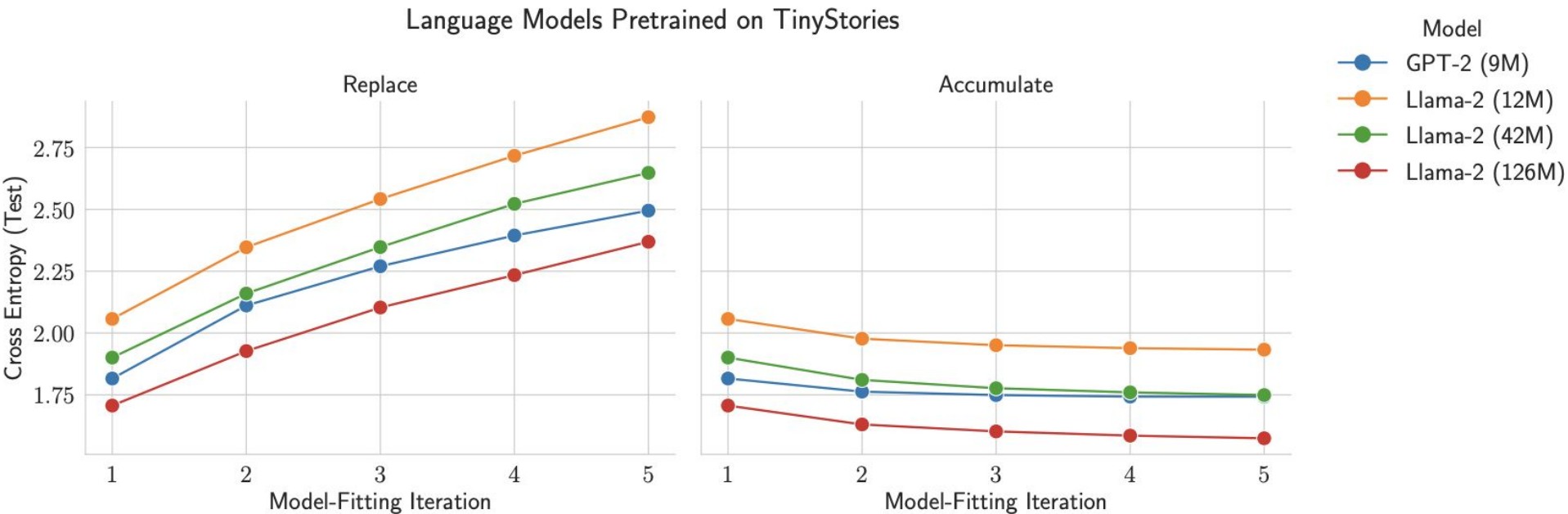
Later models are producing text the original finds almost trivially predictable. They've retreated onto the high-probability core of the distribution and are recycling it. The tall narrow spikes are indicative of a handful of sequences dominating.

Keep 10% data every generation. the same leftward drift happens but much weaker. The hump keeps its shape and the tail survives. Consistent with WRAP. The mixture is better than the parts.

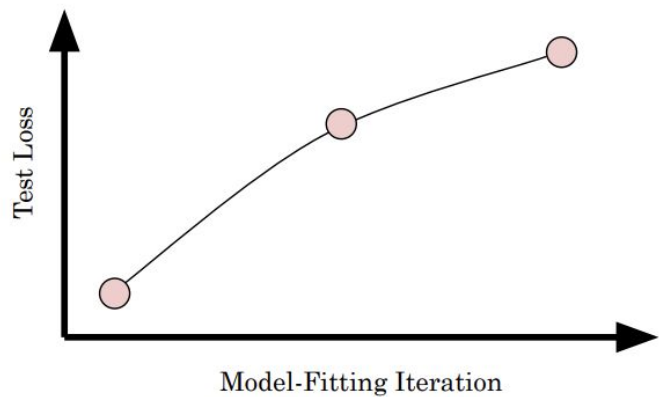
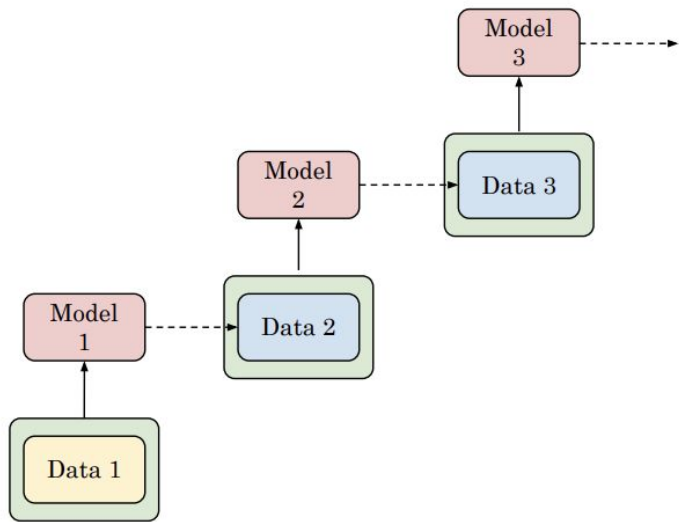
Can We Avoid Model Collapse?

Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data

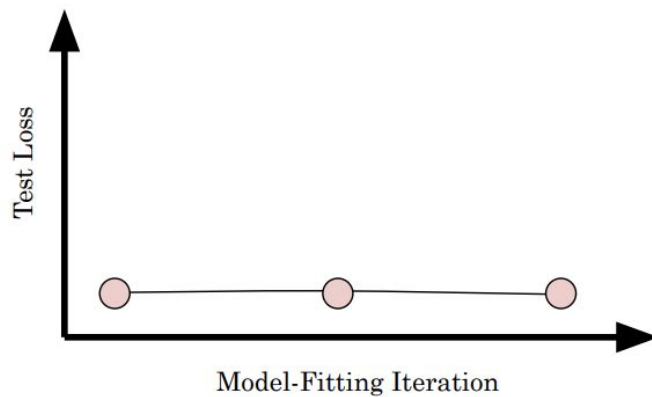
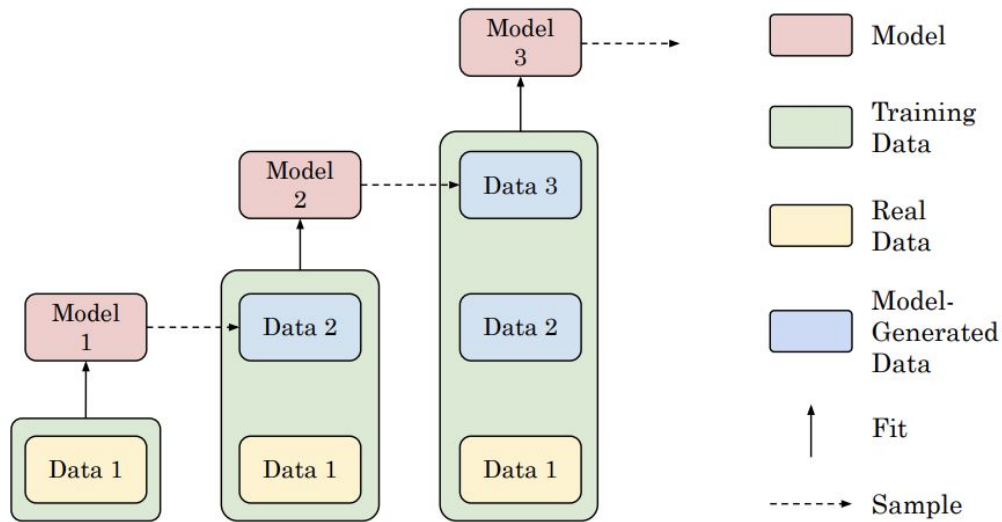
- Instead of having new data replace old data, what if we accumulated instead? Empirically, we find that accumulating the successive generations of synthetic data alongside the original real data avoids model collapse! This is consistent with the right-side figure on the previous slide.



Replace Data



Accumulate Data



A stylized illustration of a robot standing in a digital environment. The robot is white with a black visor and is positioned in the center. It is surrounded by several tall, dark server racks with glowing blue and green lights. The background is a light blue gradient with glowing green lines and dots, suggesting a network or data flow. The overall aesthetic is futuristic and technological.

Evaluation

Personally, I never evaluate because evaluation is a sign of a lack of confidence in your generation and curation.

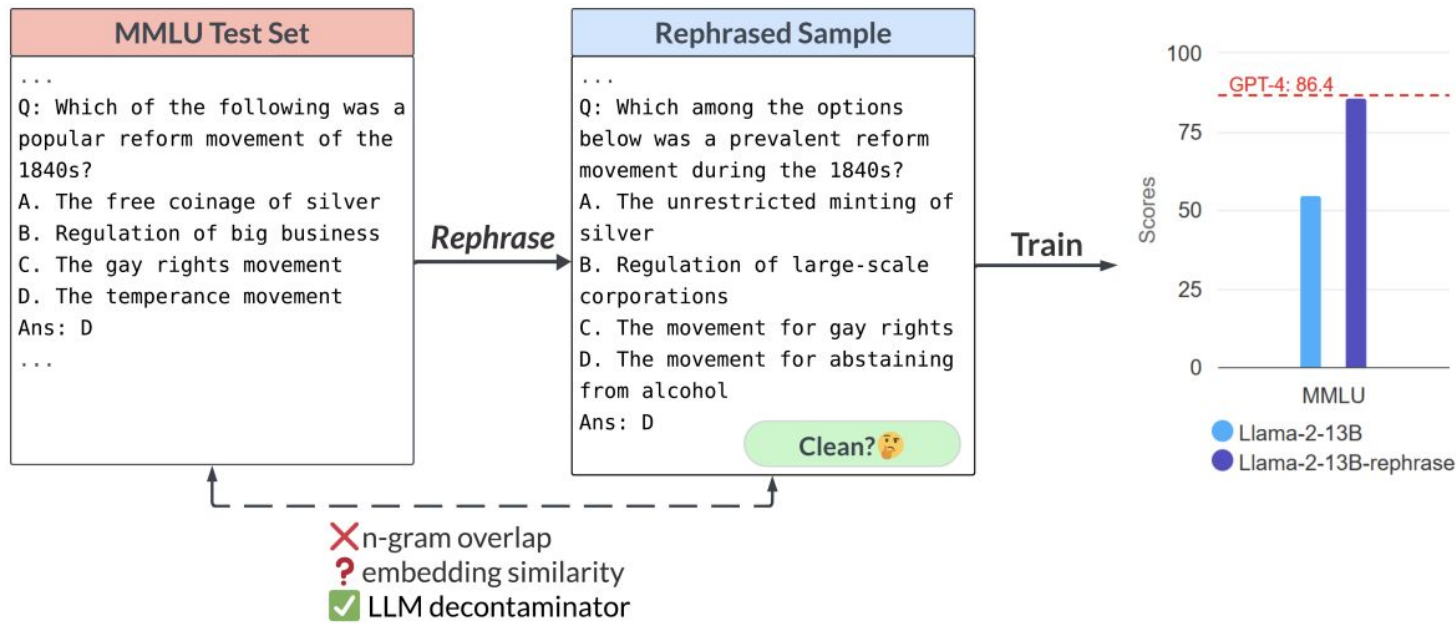
Again, how do we know any of this worked?

- A bit of a pattern emerges. DCLM, FineWeb, WRAP, Self-Instruct, model collapse. You train a model, you score a benchmark. Over and over. This is the ablation logic the entire field runs on: make a change, see the benchmark score move.
- We arrive again at benchmark decontamination. The one stage that's technically part of the curation pipeline that no one is incentivized to do correctly.
- Models score better on datasets released before their training data was collected than on ones released after. Could it be a capability gap? Maybe. Could it also be a leakage warning? Maybe.
- Can we trust public benchmarks? Can we trust reported model scores? Are our current methods of evaluation sufficient? How do we decontaminate?

Benchmark Decontamination

Rethinking Benchmark and Contamination for Language Models with Rephrased Samples

- Concerns raised about trustworthiness of public benchmarks due to potential contamination in pre-training or fine-tuning datasets.
- Most decontamination efforts: string matching (n-gram overlap)
- Simple variations of test data (paraphrasing, translation) can easily bypass this.
- Embedding similarity was attempted too: hard to find a good sim threshold.



A 13B model can easily overfit a test benchmark and achieve drastically high performance, on par with GPT-4.

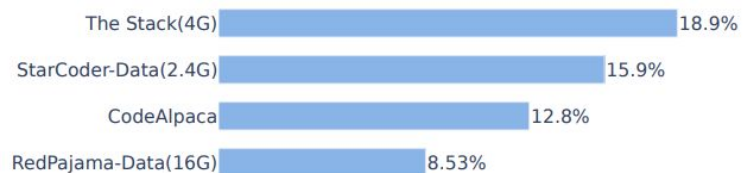


Figure 3: The contamination percentage of HumanEval benchmark in each training dataset. Note that The Stack (4G), StarCoder-Data (2.4G), and RedPajama-Data (16G) are subsets.

EXAMPLE 1 (GSM-8K REPHRASED SAMPLE)

Original Test Case

Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

Rephrased Test Case

Janet's ducks produce 16 eggs each day. She consumes three of them for her morning meal and uses four to bake muffins for her friends daily. The remaining eggs are sold at the daily farmers' market for \$2 per egg. What is the daily amount in dollars that she earns at the farmers' market?

The paper seeks to investigate whether simple variations of test sets included in the training set could affect the resulting benchmark performance.

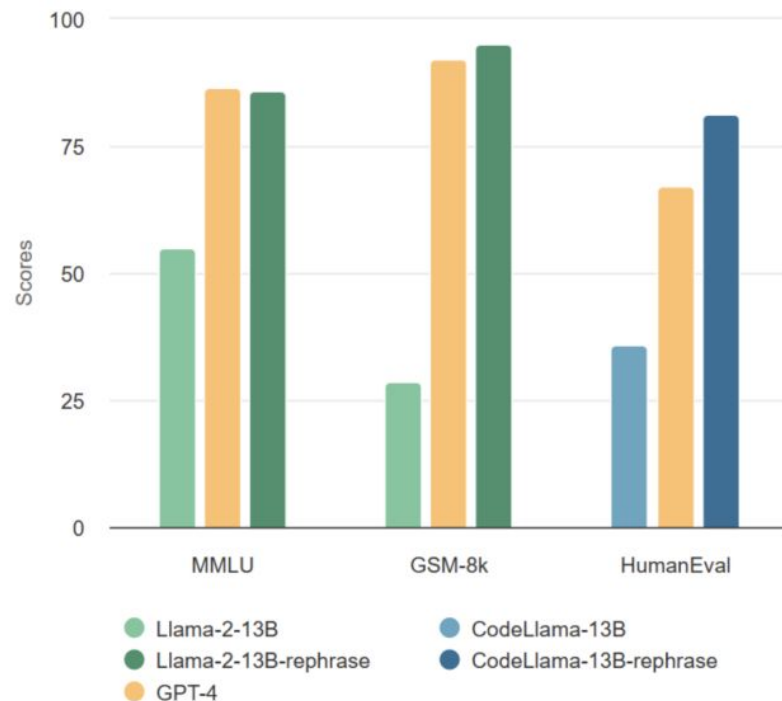
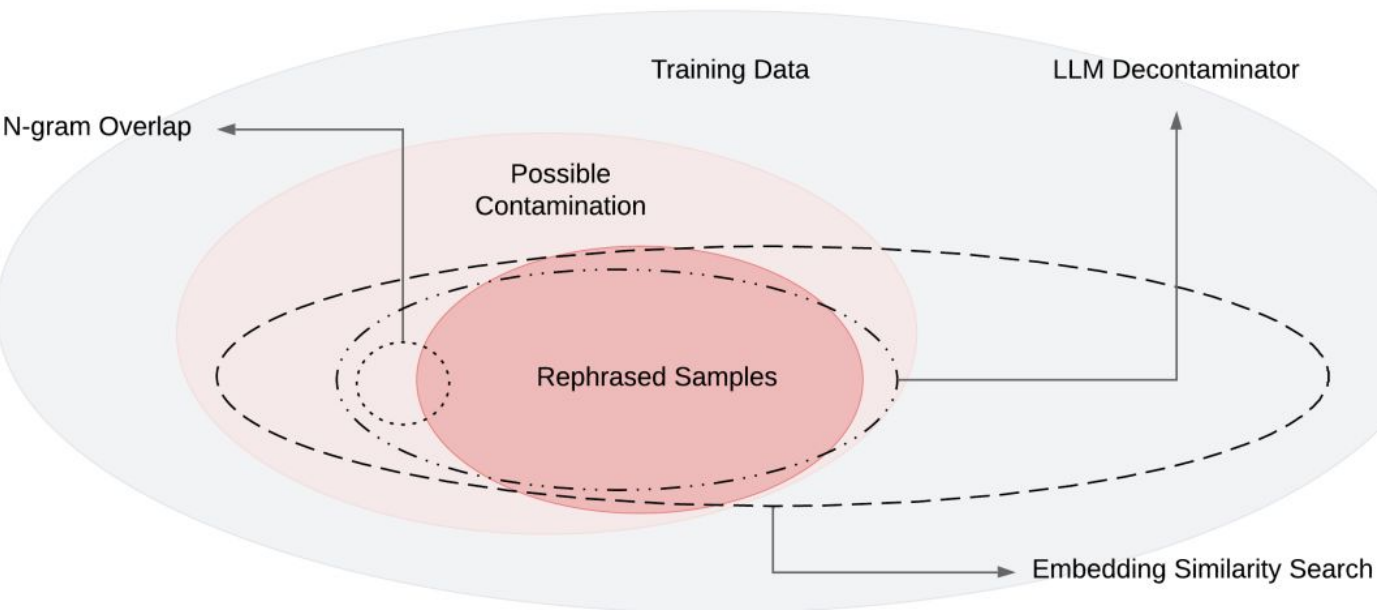


Figure 2: After fine-tuned on rephrased samples, Llama 2 and CodeLlama achieve performance on par with GPT-4.

LLM Decontaminator



Embedding similarity search detects broadly but with many false positives. N-gram overlap has a limited ability to spot rephrased samples.

This method involves two steps: First, for each test case, it identifies the top-k training items with the highest similarity using the embedding similarity search. Each pair is evaluated whether they are the same by an advanced LLM, such as GPT-4. Structured prompt: train and test example. Instruct the “LLMDetector” to carry out a comparison and return either ‘True’ or ‘False’. ‘True’ if training case might be a rephrased sample of the test case.

Algorithm 2 The algorithm for LLM decontaminator

Ensure: $\text{Decontaminate}(\text{TrainSet}, \text{TestSet}, k, \text{Template})$

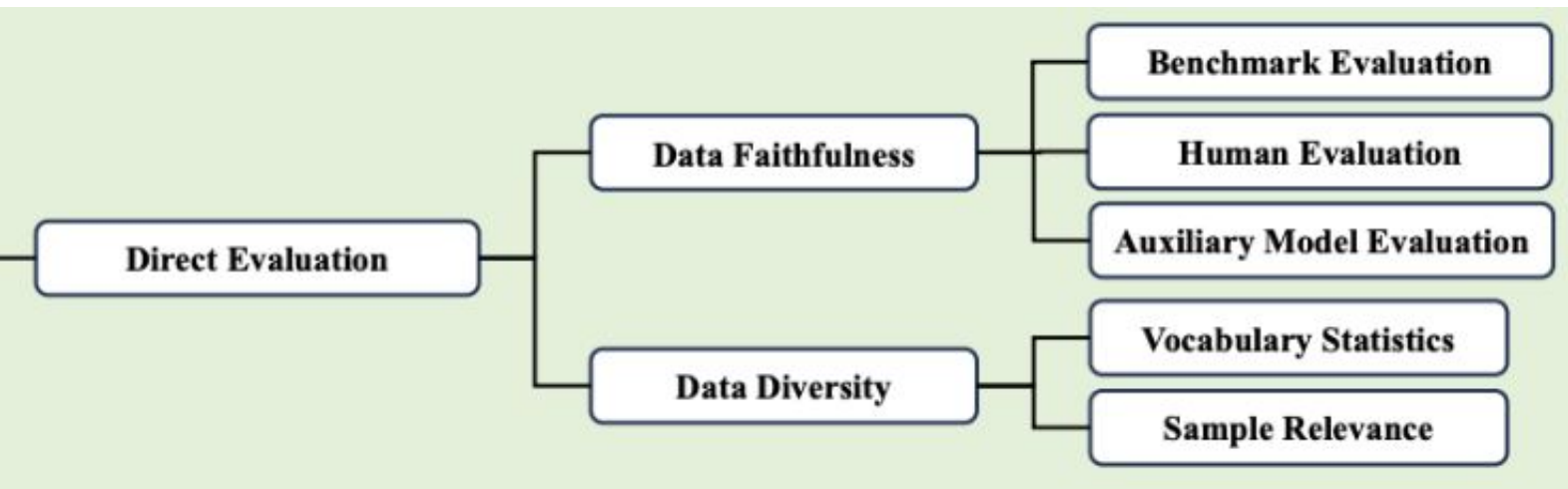
```
1:  $\text{Contamination} \leftarrow \emptyset$ 
2: for  $t$  in  $\text{TestSet}$  do
3:   for  $c$  in  $\text{TopKSimilarity}(\text{TrainSet}, t, k)$  do
4:      $s \leftarrow \text{LLMDetector}(\text{Template}, t, c)$ 
5:     if  $s = \text{True}$  then
6:        $\text{Contamination} \leftarrow \text{Contamination} \cup \{(t, c)\}$ 
7:     end if
8:   end for
9: end for
10: return  $\text{Contamination}$ 
```

Training Set	Benchmark	Size		Rephrased Samples	Percentage (%)
		Train Set	Test Set		
The Stack (4G subset)	HumanEval	500k	164	31	18.9
StarCoder-Data (2.4G subset)	HumanEval	500k	164	26	15.9
CodeExercise-Python	HumanEval	27k	164	26	15.9
CodeAlpaca	HumanEval	20k	164	21	12.8
RedPajama-Data-1T (16G subset)	HumanEval	1625k	164	14	8.5
Evol-Instruct-Code	HumanEval	78.3k	164	13	7.9
rossetacode	HumanEval	4.26k	164	4	2.4
MATHInstruct	MATH Test	262k	5000	769	15.4
MATH Train	MATH Test	7.5k	5000	79	1.6
FLAN CoT	MMLU	184k	14042	76	0.5
WizardLM-Evol-Instruct	MMLU	143k	14042	75	0.5

Some interesting related work on a the temporal version of this leakage (while using LLMs on a temporal task like stock sentiment signal generation, leakage can occur by virtue of the training corpus having information about certain assets during an assumed chronologically valid test period: [Temporal Leakage in LLM Backtesting: Measurement, Validation, and Adjusted Scores](#)

Other Evaluation Methods: Short Notes

- **Data Faithfulness**
 - Ground-truth comparison : automatic scoring where reference labels exist
 - Human evaluation : experts mark a sample; accuracy scales with sample size, so does cost
 - Auxiliary model evaluation : a model replaces the expert; cheaper, broader coverage
 - Information extraction : pull the relevant spans first, so the judge model fits them in context
- **Data Diversity**
 - Vocabulary statistics : vocabulary size, n-gram frequency. Intuitive, semantically blind
 - Sample relevance : cosine similarity, sample distance. Captures semantic spread



Open Problems

1. Semantic decontamination doesn't scale. An embedding index over a trillion-token corpus is affordable, but LLM-judging every retrieved candidate against every benchmark item is not
2. A bigger teacher doesn't make better data. Contrary to expectation, scaling the generator does not monotonically help: 8B generators consistently beat 3B ones, but going up to 70B often produced worse synthetic data.
3. No scaling law for synthetic data, and the literature disagrees on the mixing ratio. A 1000-model study found optimal ratios converge near 30% synthetic, stable across sizes and budgets. Other 2026 work finds the optimal ratio rises with model size: around 12.5% at 3B and 25% at 6B. WRAP used 1:1. No formal theory yet.

Bibliography

Framework

Long, Wang, Xiao, Zhao, Ding, Chen & Wang. *On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey*. Findings of ACL 2024. arXiv:2406.15126

Curation

Li et al. *DataComp-LM: In Search of the Next Generation of Training Sets for Language Models*. NeurIPS 2024. arXiv:2406.11794
Penedo et al. *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale*. NeurIPS 2024. arXiv:2406.17557
Broder. *On the Resemblance and Containment of Documents*. SEQUENCES 1997.
Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (C4)*. JMLR 2020. arXiv:1910.10683

Generation — pretraining

Maini, Seto, Bansal, Kale, Schwarzschild & Kolter. *Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling (WRAP)*. ACL 2024. arXiv:2401.16380
Su et al. *Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset*. arXiv:2412.02595

Generation — post-training

Wang, Kordi, Mishra, Liu, Smith, Khashabi & Hajishirzi. *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. ACL 2023. arXiv:2212.10560
Xu, Sun, Zheng, Geng, Zhao, Feng, Tao & Jiang. *WizardLM: Empowering Large Language Models to Follow Complex Instructions (Evol-Instruct)*. ICLR 2024. arXiv:2304.12244
Taori et al. *Stanford Alpaca: An Instruction-Following LLaMA Model*. 2023.

Environments and self-play (drop if you cut slide 17)

Lambert et al. *Tulu 3: Pushing Frontiers in Open Language Model Post-Training*. arXiv:2411.15124
Zhao et al. *Absolute Zero: Reinforced Self-play Reasoning with Zero Data*. arXiv:2505.03335
Huang et al. *R-Zero: Self-Evolving Reasoning LLM from Zero Data*. arXiv:2508.05004

Model collapse

Shumailov, Shumaylov, Zhao, Papernot, Anderson & Gal. *AI Models Collapse When Trained on Recursively Generated Data*. Nature 631:755–759, 2024. doi:10.1038/s41586-024-07566-y
Gerstgrasser et al. *Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data*. COLM 2024. arXiv:2404.01413

Contamination

Yang, Chiang, Zheng, Gonzalez & Stoica. *Rethinking Benchmark and Contamination for Language Models with Rephrased Samples*. arXiv:2311.04850
Xu et al. *Benchmark Data Contamination of Large Language Models: A Survey*. arXiv:2406.04244
Li & Flanigan. *Task Contamination: Language Models May Not Be Few-Shot Anymore*. AAAI 2024. arXiv:2312.16337
Dev et al. *Test of Time: Rethinking Temporal Signal of Benchmark Contamination*. ACL 2026. arXiv:2509.00072
Jain et al. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. ICLR 2025. arXiv:2403.07974
Singh et al. *The Leaderboard Illusion*. 2025. arXiv:2504.20879

Open problems

Nait Saada, Bethune, Klein, Grangier, Cuturi & Ablin. *The Data-Quality Illusion: Rethinking Classifier-Based Quality Filtering for LLM Pretraining*. arXiv:2510.00866
Klimaszewski et al. *Is a Document Educational or Just Wikipedia-Style? Pitfalls of Classifier-Based Quality Filtering*. 2026. arXiv:2605.23721
Kang et al. *Demystifying Synthetic Data in LLM Pre-training: A Systematic Study of Scaling Laws, Benefits, and Pitfalls*. EMNLP 2025. arXiv:2510.01631
Chen et al. *On the Diversity of Synthetic Data and its Impact on Training Large Language Models*. arXiv:2410.15226
Zhu et al. *Measuring Diversity in Synthetic Datasets (DCScore)*. arXiv:2502.08512